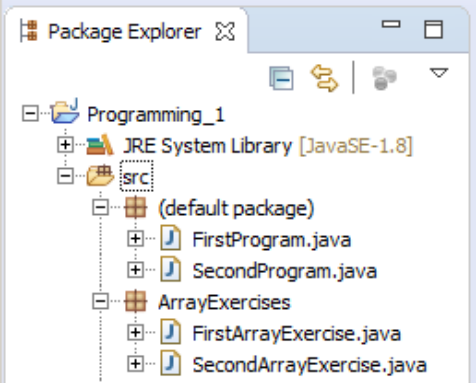
ECLIPSE QUICK START

BASIC CONCEPTS 1
How to create a new Java Project 2
* How to create a new package in the project (*optional*) 2
* How to create a new Java program 2
* How to run a Java program 2
The basic set of windows in Eclipse 2
How to use automatic code formatting to have correct indentation and style 2
How to make Eclipse to automatically correct an error in your code (Quick Fix) 3
How to generate class members (constructors, getters, setters, toString) automatically 3
How to save time and effort by using the Content assist 4
Practical code editing 5
HOW TO EXPORT AND IMPORT JAVA PROJECTS 5
HOW TO USE THE DEBUGGER 6

BASIC CONCEPTS

When you start ***Eclipse*** for the first time, it asks you to select a folder as workspace.

A ***workspace*** is an Eclipse-managed container where you can create many ***projects***. Each project can be of different type (Java Project, Dynamic Web Project, JavaScript Project etc.). In a **Java Project**, you can create Java ***classes*** in the **src** folder.



A very simple stand-alone *Java program* consists of one Java class. Each Java class belongs to a Java ***package***. You can create any number of packages in the **src** folder. The ***default package*** is an unnamed package. If you have not created any packages of your own, then your classes automatically belong to the default package.

Java provides the default package for convenience when you are just beginning Java development. In professional programming, you should always use named packages.

In the example, there are two classes in the *default package* and two classes in the *ArrayExercises* package.

Eclipse lists your projects in the **Package Explorer** window. If Eclipse shows the **Project Explorer** window instead, then the *current perspective* is not the Java Perspective.

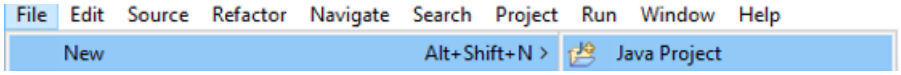
A **perspective** is a visual container for a set of views, editors (parts), dropdown menu contents etc. You can change the current perspective to **Java Perspective** as follows: Window > Open Perspective > **Java**

Behind the scenes, Eclipse creates the workspace - project - src - package folder hierarchy on the disk. In addition, Eclipse creates other folders for its internal use in the workspace folder. **NB!** You should never directly manipulate the workspace content outside of Eclipse. You should add new files, import and export files etc. only in Eclipse.

Key concepts: Eclipse → workspace → project → package → Java class
perspective in Eclipse, Java perspective, Package Explorer, default package vs named package

Note: You can create several workspaces in Eclipse. To create a new workspace or use another existing workspace, you must *switch a workspace* in Eclipse as follows: File > Switch Workspace. Eclipse restarts when you create a new workspace or change to another existing workspace.

How to create a new Java Project



File > New > **Java Project** > *Enter the project name* > **Next** > *Unselect 'Create module-info.java file '* > **Finish**

NB! If you do not see **Java Project** as the first item in the drop down list as above, then you must change the current perspective to **Java Perspective** as follows: **Window** > Open Perspective > **Java**

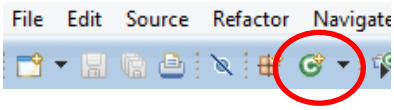
* How to create a new package in the project (*optional*)

Right-click on the **src** folder name or project name > **New** > **Package**

* How to create a new Java program

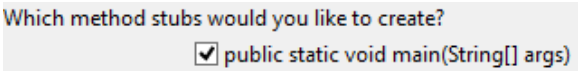
1. Right-click on the package name (or the **src** folder name, or the project name) > **New** > **Class**

   OR Click on the following icon:



Then,

2. Enter the name of the class to be created
3. **Select** the method **public static void main(String[] args)** to be created. Unselect the other options.



4. Finally, click on **Finish**

* How to run a Java program

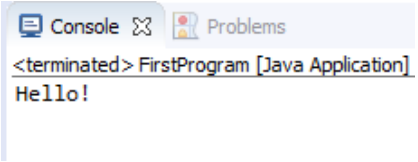
Alternative ways of running the Java program in the active tab of the editor include the following:

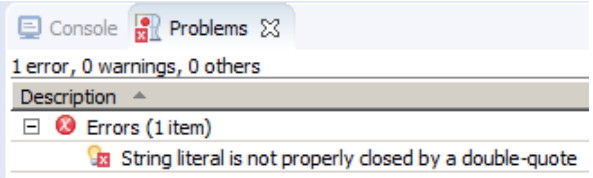
* Run > **Run**
* Press **Ctrl** + **F11**
* Click on the Run icon

The basic set of windows in Eclipse

The **Package Explorer** Window, **Java Editor** window, **Console** Window and **Problems** Window are always needed in Java programming.

The Java Editor includes the following features: ***syntax highlighting***, ***content/code assist***, ***code formatting***, ***import assistance***, ***Quick fix***, and integrated ***debugging*** features. The Console Window shows ***program output*** and ***run-time error messages***. The Problems Window shows ***compile-time error messages*** and ***warnings***.





If some window is not visible...

* Select **Window** > Show view > Then, select the name of the window to be shown.

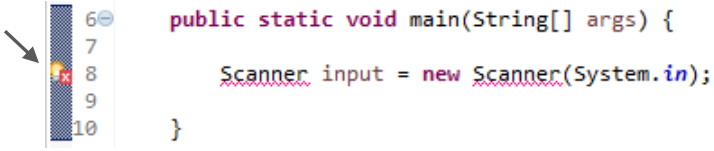
How to use automatic code formatting to have correct indentation and style

* Press **Ctrl + Shift + F**

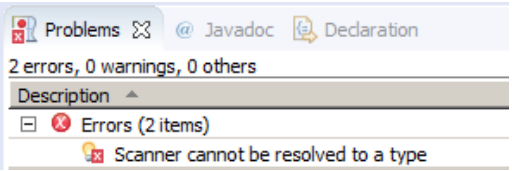
Use this often !

How to make Eclipse to automatically correct an error in your code (Quick Fix)

Eclipse can suggest ***automatic corrections*** to most of such problems that it shows underlined in red in your code.



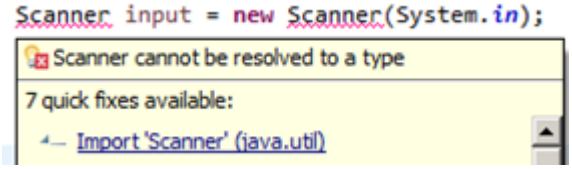
The *small yellow light bulb* in the editor marker bar indicates that there is a Quick Fix available. The light bulb is also shown in the **Problems** window where Eclipse shows a ***list of all errors*** and warnings.



To see the Quick Fix suggestions, you can alternatively

1. Click on the small yellow light bulb.
2. Place the cursor on the text that is underlined in red.
3. Right-click on the error message in the **Problems** window and select **Quick Fix**.

When you click on one of the Quick Fix suggestions, Eclipse makes the correction in your code.



When you click on the suggestion "Import 'Scanner' (java.util)", Eclipse adds the following line of code to the source text: `'import java.util.Scanner;'`

Hint	**Let Eclipse to add the import directives** Instead of always writing all the required import directives yourself, you can ***use the Quick Fix feature*** as above and let Eclipse to add the import directives

NB! Close other unrelated projects to get rid of unrelated error messages. When you are building a program, Eclipse shows all the error and warning messages from all the open projects in the **Problems** window.

How to generate class members (constructors, getters, setters, toString) automatically

A typical *class for objects* might contain several fields, one or more constructors, several getters and setters, and the toString method. For example, if we write a simple class for Course objects with three fields (e.g. course code, course name and credits), the total number of lines of code to be written is 30 – 35. In Eclipse, you can ***automatically generate 90 % of the code*** of the Course class. First, you write only *three lines of code* to define the three fields. Then, you let Eclipse to generate constructors, getters and setters, and the toString() method.

In Eclipse toolbar, for example, you can select

Source > Generate Getters and Setters...
Source > Generate toString()...
Source > Generate Constructor using Fields...

How to use the help feature

To get help on a window (Editor, Package explorer, Console, Problems etc.) in Eclipse,

* Place the cursor to a window > Press **F1**

How to save time and effort by using the Content assist

Content assist is a general term a set of code editing features including code completion, quick info, member lists and code snippets/templates. Microsoft calls these features *IntelliSense*. Code assist helps developers to write code more effectively and avoid simple some simple errors. You can trigger Content assist in the Editor window by pressing **Ctrl + Space** or by typing a dot (see below).

Code completion

When you type one or more characters and press **Ctrl** + **Spacebar**, Content assist shows a context-sensitive popup list of possible existing completions (identifiers, keywords etc.) to the characters you have already typed.

Quick info

When you place the cursor on an identifier (name of a class/method/variable), Content assist shows the related short description of the item. Eclipse generates these descriptions based on the original *javadoc* comments on the referenced items. You can also see your own javadoc comments (the text written within /** and */).

Member list

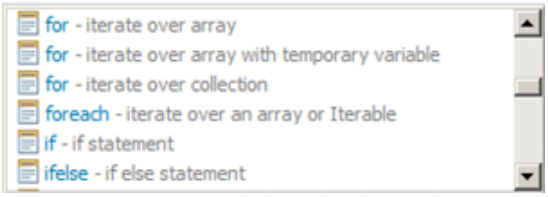
When you type a dot after a class name or reference variable name, Content assist shows a popup list where you can select the desired class member etc.

Code snippets / templates

Eclipse provides dozens of pre-defined ***Java code snippets***. In addition, you can create your own code snippets. When you press **Ctrl** + **Spacebar**, Eclipse shows a popup list where you can select a code snippet to be inserted into your source text. The list of snippets depends on the context. Eclipse determines the context based on the current location of the cursor. In addition, you can pre-filter the list of snippets by first typing one or more letters (or the whole word) before you press Ctrl + Spacebar.

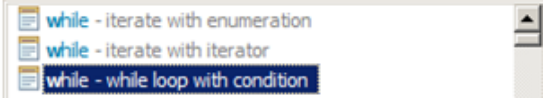
If the cursor is ***inside a method definition***, the popup list contains entries like below.

for - iterate over array (several variations)
for - iterate over collection
if statement
if-else statement
try-catch block
while loop with condition



For example, a fast way to **create a *while statement*** is as below.

1. Type **w**
2. Press **Ctrl** + **Spacebar**
3. Select '**while -** while loop with condition'

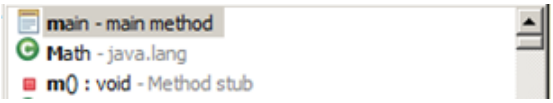


A fast way to write '**`System.out.println();`**' is as below.
1. Type **syso**
2. Press **Ctrl** + **Spacebar**

If the cursor is within a class definition, but ***outside of a method definition***, the popup list contains a different list of code snippets.

For example, a fast way to **insert a missing *main method*** is as below.

1. Type **m**
2. Press **Ctrl** + **Spacebar**
3. Press Enter to select '**main -** main method'



If the code layout is not ok after inserting a snippet, you can adjust the layout by pressing **Ctrl** + **Shift** + **F**

The Java Editor can also **automatically find the matching bracket**.

* Select an opening or closing bracket in your code, then
  Select **Navigate** > Go To > **Matching bracket** or Press **Ctrl** + **Shift** + **P**

How to open and close projects

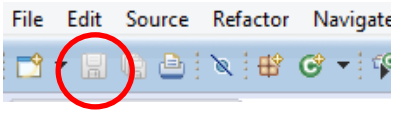
Open an existing Java project
* In Package Explorer, select the Java project name > Right-click > **Open Project**

Close a Java project
* In Package Explorer, select the Java project name > Right-click > **Close Project**

Close all unrelated Java Projects
* In Package Explorer, select the Java project name > Right-click > **Close Unrelated Projects**

Practical code editing

Save the Java code from the Editor Window
Alternative ways of ***saving the Java code*** in the active tab of the editor include the following:
* File > Save
* Press **Ctrl + s**
* Click on the following icon:



Shift text
* First, select the text block to be shifted, then Select **Source** > **Shift Left**
* First, select the text block to be shifted, then Select **Source** > **Shift Right**

Toggle a comment
* First, select the code block, then Select **Source** > **Toggle Comment**

How to change font size in Eclipse

* Window > Preferences > Appearance > Colors and Fonts > Java > **Java Editor Text Font** > Edit...
* Window > Editor > Zoom In
* Window > Editor > Zoom Out

Drag-and-drop or copy/paste single files between Eclipse and the host system

* If you are using your own named packages in your Java project, then drag (or copy/paste) the .java file from the host system *to the **src** root folder first*. Then, drag the file from the **src** root folder to the correct package in your project. This way, Eclipse is able to automatically insert correct package definitions in your .java file.

HOW TO EXPORT AND IMPORT JAVA PROJECTS

NB! If you want to make a copy of your Java project and take it somewhere else, then ***always export the project in Eclipse as explained below***. If you directly copy a Java project folder in the host system without Eclipse, then the copy (or Zip file) will miss the .classpath and .project files etc. and it cannot be properly imported to another workspace.

Export a project to a Zip file

File > **Export...** > General > **Archive File** > *Select the project you want to export*
> **Browse ...** > *Select the folder where you want to save the Zip file*
> **To Archive file:** > *Enter a name for the zip file to be created* > **Save** > **Finish**

Import a project from a Zip file

File > **Import...** > General > **Existing Projects into Workspace**
> **Select archive file:** > **Browse ...** > *Select the Zip file to be imported* > **Open** > **Finish**

HOW TO USE THE DEBUGGER

Open the debugger

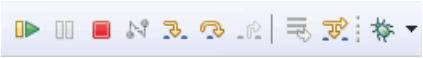
* Window > Open Perspective > Debug
* Alternatively, Click on > Debug

Set or remove a breakpoint

* Right-click in the left margin in the Java editor > Toggle Breakpoint
* Alternatively, Double-click on this position

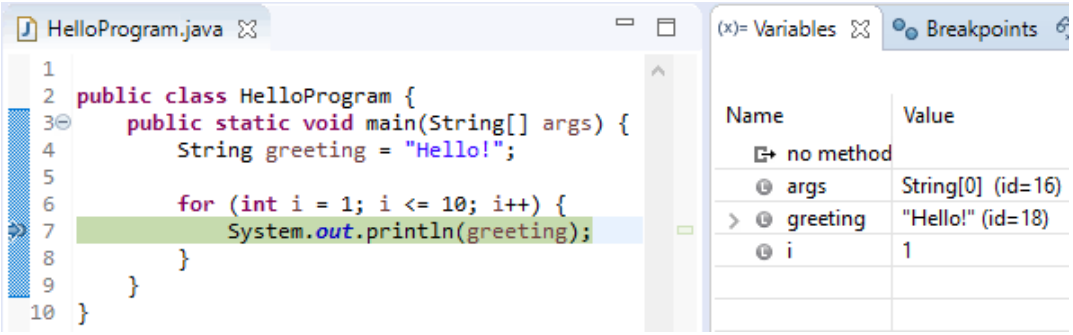
Eclipse marks the breakpoint with a *small green dot* in the left margin.
You can have one or more breakpoints in your code.

Start debugging and step through code



	F11	Start debugging	
	F5	**Step into**	Advance to the next line in the method, unless on a method call, in which case enter the method being called.
	F6	**Step over**	Advance to the next line in the same method.
	F8	**Resume**	Continue running to the next breakpoint.
	Ctrl + F2	Terminate program execution	
	Ctrl + F11	Start without debugging	

When the code has stopped at a breakpoint, you can examine its variables in the **Variables** tab. You can also hover the mouse pointer on a variable name in the code to see its current value. The debugger shows values of only those variables that are visible in the current scope. In addition, you can change values of variables in the Variables tab.



Close the debugger perspective

* Window > Open Perspective > **Java**
* Alternatively, Click on